CS 433 Operating Systems — Programming Assignment 2

tsum: doing one job with many threads

CS 433 Operating Systems · CSU San Marcos · Fall 2026 · Dr. Reza Ramezan

Released Monday, October 5, 2026 **Due** Friday, October 16, 2026, 11:59 PM, on Canvas **Weight** 100 points **Work** Alone, or in a fixed group of any size (see §10) **Turn in** Canvas, one .tar.gz named LastName_FirstName_PA2.tar.gz

> **This assignment is the thread half of the midterm.** The midterm on **Monday, October 12** covers multi-process programming from PA1 and multi-thread programming from this assignment. Parts A and B are the material the exam asks about, so do those two parts in the first week, before the exam. Parts C and D are due with everything else on October 16.

---

1. What you are building

Four small things, in one directory:

Part	What	Points
A	tsum — sum an array with T threads, no locks	50
B	Four broken threaded programs. Find the bug, fix it, name it	20
C	tcount — show that the kernel really does see your threads	10
D	analysis.md — measure the speedup, measure the serial fraction, predict the ceiling	20

Everything is Chapter 4 material. **You will not need a mutex, a semaphore or a condition variable anywhere in this assignment**, and you should not use one. If you reach for a lock in Part A, stop and re-read section 4.2 — the design avoids the need for one. Locking comes in PA4, after we cover Chapter 6.

2. Learning objectives

When you are done you should be able to:

1. Create and join threads with pthread_create and pthread_join, and say what each argument is for.
2. Hand each thread its own arguments safely, and explain why passing the address of a loop counter does not work.
3. Divide a data-parallel job into disjoint ranges so the workers need no locks, and join every worker before main reads and combines their results.
4. Read the return value of a Pthreads function correctly — it is not the −1-and-errno convention you learned for system calls.
5. Measure speedup, measure the serial fraction of a program, and use Amdahl's law to predict the ceiling before you buy more cores.
6. Confirm from outside a process that a Pthreads thread is one-to-one with a kernel-schedulable entity on Linux and macOS.

3. Why anyone cares

The reason to use threads is almost never elegance; it is that you have more than one core and a job that can be cut into pieces. Two things decide whether that works. The first is whether the pieces are genuinely independent — if they are, you need no locks, and the fastest threaded code is the code that never synchronises. The second is Amdahl's law, which says the part you could not parallelise sets a hard ceiling on what any number of cores can do for you. Part D makes you measure that ceiling on your own machine rather than take it on faith, and the number usually surprises people.

---

4. Part A — tsum (50 points)

4.1 What it does

tsum runs two phases.

Phase 1, serial. Fill an array of n doubles with x[i] = sin(i). This phase is single-threaded on purpose. **Do not thread it.** It is the serial fraction that Part D measures.

Phase 2, parallel. Compute the sum of work(x[i]) over the whole array, using T threads. work() is given to you in tsum.c. **Do not change work()** — the grader compares your total against the reference, and editing work() changes the answer.

4.2 The design: partition, do not lock

Thread k owns the half-open range [lo, hi):

```
lo = (long)k * n / nthreads
hi = (long)(k + 1) * n / nthreads
```

Write it exactly this way, with the multiply before the divide, and the ranges tile [0, n) with no gap and no overlap even when nthreads does not divide n. The range widths differ by at most one and together cover every element — with n = 11 and T = 3 they are 3, 4, 4, not 3, 3, 5. Check that for yourself with n = 10, nthreads = 3.

Each thread adds up its own range and stores the result in **its own slot**. No two threads write the same memory, and no thread reads a slot another thread is writing. That is why there is no race and why you need no lock. When all the threads have been joined, main adds the T partial sums together.

Accumulate inside the thread into a **local** double, then assign it to the struct field once at the end. Accumulate locally and store once: that is the pattern being asked for, and it does not depend on how any particular compiler allocates registers.

4.3 Command line

./tsum [--threads T] [--n N] [--reps R]

Flag	Meaning	Default
--threads T	number of threads in phase 2	1
--n N	array length	20 000 000
--reps R	run both phases R times, report the fastest	3

Reject T < 1, N < 1 or R < 1 with a message on stderr and exit status 2. An unrecognised flag is also exit status 2. Out of memory is exit status 1.

--reps reports the **fastest complete repetition**, not the average and not the fastest fill paired with the fastest sum from a different run. A slow run means something else on the machine wanted the CPU; that is noise about the machine, not information about your code.

wall_s is fill_s + sum_s for that one repetition. It covers the two timed phases only — not the initial allocation or the printing — so the serial fraction you compute from it is an estimate under the two-phase model, and thread creation, joining and the final reduction all sit inside sum_s.

4.4 Output format

Exactly these seven lines, in this order, on stdout:

```
threads   4
n         20 000 000
reps      3
total     48 766 030.903176
fill_s    0.0737
sum_s     0.0518
wall_s    0.1255
```

total with %.6f, the three times with %.4f. The grader reads these lines, so do not add, remove or reorder them.

4.5 Your total will not be bit-identical to anyone else's, and that is correct

Floating-point addition is **not associative**: (a + b) + c and a + (b + c) can differ in the last bits. Changing the number of threads changes which elements get added together first, so the total shifts slightly:

threads	total
1	48 766 030.903168
2	48 766 030.903176
4	48 766 030.903176
14	48 766 030.903175

This is not a bug and you must not try to fix it. The grader accepts any total within a relative tolerance of 1e-9 of the reference. Part D asks you to explain it.

What you **must** do is add the partials in **thread order** — 0, 1, 2, … — rather than in whatever order the threads happen to finish. Thread order is reproducible; completion order is not, and a result that changes between runs of the same binary is a result you cannot debug.

4.6 Error handling

pthread_create and pthread_join **return an error number on failure**. They do **not** return −1 and they do **not** set errno:

```
int rc = pthread_create(&tid[k], NULL, sum_range, &args[k]);
if (rc != 0) {
    fprintf(stderr, "tsum: pthread_create for thread %d: %s\n", k, strerror(rc));
    ...
}
```

If a create fails partway through, join the threads you did create before you exit. Do not join a pthread_t that was never filled in.

4.7 REQUIRED — the checklist

- ☐ make builds tsum with **no warnings** under -Wall -Wextra
- ☐ Phase 1 is single-threaded
- ☐ work() is unmodified
- ☐ Every thread has its **own** argument struct
- ☐ Ranges tile [0, n) exactly, verified for a case where T does not divide n
- ☐ No mutex, semaphore, condition variable or atomic anywhere
- ☐ Return values of pthread_create and pthread_join are checked as error numbers
- ☐ Partials summed in thread order
- ☐ Seven output lines, exact format
- ☐ ./tsum --threads 1 and ./tsum --threads 8 agree to within 1e-9 relative
- ☐ No memory leaks: everything malloced is freed
- ☐ make clean removes every binary

4.8 Where the 50 points are

	Points
Correct total at T = 1	10
Correct total at T = 2, 4, 8, within tolerance	10
Ranges tile exactly, including the remainder case	8
Own argument struct per thread; no shared-counter bug	8
Error numbers checked correctly on create and join	6
Output format exact	4
No locks; no leaks; clean build under -Wall -Wextra	4

4.9 STRETCH — extra credit, up to +6 (your total is still capped at 100)

- ✦**3** Add --chunk C, which hands threads rotating blocks of C elements instead of one contiguous range each, and report in analysis.md whether it changes the timing on this workload and why you think so.
- ✦**3** Allocate the array with calloc instead of malloc, measure phase 1 again, and explain the difference you see in terms of what the kernel does with a fresh page. Report the numbers.

---

5. Part B — four broken programs (20 points)

bugs/ holds four programs. Each one compiles and each one is wrong.

File	Advertised symptom
bug1_args.c	Should print indices 0–3 once each. Prints wrong, repeated, or out-of-range numbers, and differently on each run
bug2_nojoin.c	Should print total 4000. Prints less
bug3_stack.c	Should print 100, 200, 300, 400. Crashes, or prints garbage
bug4_errno.c	Reports success when creation failed, then prints a result it never received. Run it with CS433_FAIL_CREATE=1 too, as the file explains

For each one:

1. **Run it first** and see the symptom with your own eyes. Run bug1 several times. These are defects, not guaranteed behaviours: a broken program may print the same wrong answer every time, or occasionally the right one. If what you see differs from the table, report what you actually saw.
2. **Fix it.** Keep the shape of the program. Do not delete the interesting part to make the symptom disappear — bug4 tells you which line to leave alone, and bug3 wants the value to outlive the thread, not the printing to be removed.
3. **Name the defect** in one sentence in bugs/ANSWERS.md, plus one sentence on what you changed. A sentence that says "I added a join" scores less than one that says what was wrong with not having it.

One of these four, and only one, is visible to the compiler. Build with -Wall -Wextra first and read what it says: it hands you one of the bugs outright. The other three compile silently, which is the real lesson — a clean build is not evidence that threaded code is correct. The three silent ones are found by running the program and by reading it, not by the compiler.

Points: 4 per program for a correct fix, 1 per program for correctly naming the defect.

---

6. Part C — tcount (10 points)

tcount.c is written for you. It prints its pid, waits about **3 seconds**, creates T threads that stay alive for about **6 seconds**, then exits. There is no prompt: both windows are timed, so have your counting command ready in the second terminal before you start it.

Run it in one terminal and count its threads from another while it waits.

On Linux (the course server):

```
./tcount --threads 4 &
ls /proc/<pid>/task | wc -l
ps -o nlwp= -p <pid>
```

On macOS:

```
./tcount --threads 4 &
ps -M <pid> | tail -n +2 | wc -l
```

Record in analysis.md:

1. The thread count **before** the threads are created, and **after**.
2. The difference, and whether it matches the number you asked for.
3. One or two sentences: does this support the claim that Pthreads on this system is one-to-one, or many-to-one? What would you have seen instead if the mapping were many-to-one?

Do this on **two** systems if you can — the course server and your laptop — and say whether the answers differ.

Points: 6 for correct before/after numbers with the command you used, 4 for the reasoning.

---

7. Part D — the measurements (20 points)

In analysis.md.

7.1 The speedup table

Run tsum at 1, 2, 4, 8 threads, and once more at the core count of the machine. Get the logical CPU count with nproc on Linux or sysctl -n hw.ncpu on macOS. Fill in:

threads	fill_s	sum_s	wall_s	sum-phase speedup	whole-program speedup
1				1.00	1.00
2					
4					
8					
cores					

Speedup is always measured against your own 1-thread run, not against anyone else's number and not against a theoretical ideal.

7.2 The serial fraction, measured

Amdahl's law says that if a fraction **S** of the work is inescapably serial, then no number of processors gets you past **1 / S**.

Do not guess **S**. You measured it: at one thread it is

S = fill_s / wall_s

Report **S** and report the ceiling **1 / S** for your machine.

7.3 The four questions

1. What is your measured **S**, and what ceiling does it imply?
2. What whole-program speedup did you actually reach at the core count? How far below the ceiling is it, and name one reason for the gap.
3. Compare your **sum-phase** speedup with your whole-program speedup and explain the difference you measured in one or two sentences. Which of the two would a user of this program notice?
4. Why does total change in its last digits when you change the thread count, and why is that not a bug? (Section 4.5.)

If your machine has performance cores and efficiency cores — recent Apple silicon, and most recent Intel laptops — you may find that going from 8 or 10 threads to the full core count makes things **slower**. If you see that, report it and say what you think is happening. It is a real effect and noticing it earns full credit on question 2.

Points: 6 for the completed table, 4 for a correctly measured S and ceiling, 10 for the four questions.

---

8. Getting started

```
unzip PA2_starter.zip
cd PA2
make
./bugs/bug1_args          # see Part B misbehave before you fix anything
./tsum --threads 1 --n 2 000 000
```

A fresh make prints **three warnings**, and all three are expected: work and sum_range are unused in tsum.c until you implement them, and bugs/bug3_stack.c draws a -Wreturn-stack-address warning, which is the compiler handing you one of the Part B bugs for free. All three disappear as you do the work. Any **other** warning is yours, and make must be silent by the time you submit.

Use a small --n like 2 000 000 while you are debugging, and the default 20 000 000 only for the Part D timings. Timing numbers from a laptop on battery are not worth much; plug in, or use the course server.

gcc and clang both work. You must link -pthread (the Makefile does) and -lm for the math functions.

9. Checking your own work

```
make clean && make            # must be warning-free
./tsum --threads 1  | grep total
./tsum --threads 8  | grep total     # same to 1e-9 relative
./tsum --threads 3 --n 10             # does the remainder case tile correctly?
./tsum --threads 0                    # must exit 2 with a message
grep -nE 'mutex|semaphore|cond_|atomic' tsum.c   # only the file's own comment may match
```

On Linux, if valgrind is available:

```
valgrind --leak-check=full ./tsum --threads 4 --n 100 000
```

Expect “no leaks are possible”. helgrind should report no races in Part A; if it reports one, two of your threads are touching the same memory and your partition is wrong.

10. Submitting

One file on Canvas: LastName_FirstName_PA2.tar.gz, built from a clean tree.

```
make clean
cd ..
tar czf Ramezan_Reza_PA2.tar.gz PA2/
```

It must contain tsum.c, tcount.c, Makefile, the four fixed programs in bugs/, bugs/ANSWERS.md, and analysis.md. Do not submit binaries, object files, or a .git directory.

If you are working in a group. A group submits **one** copy. Name the archive after whichever member uploads it, and put **every** member's name at the top of analysis.md, in bugs/ANSWERS.md, and in the header comment of tsum.c. Each of the other members uploads a single file called PARTNER.txt containing one line:

```
Submitted by: Lastname, Firstname
```

Everyone in the group receives the same grade. Groups are fixed for the semester; changing later needs my approval in advance. **Every member must be able to explain every line of the submission** — and note that the midterm is individual and draws on Parts A and B, so a part you did not write yourself is a part you will meet again alone.

11. Collaboration policy

You may work alone or in a fixed group of any size, and within your group you share everything: code, debugging, the written answers. The whole group is responsible for all of it.

Across groups it is different. Talk about ideas with anyone, but do not look at another group's tsum.c or bugs/*.c, and do not show anyone yours. If you discuss the assignment with someone outside your group, name them at the top of analysis.md; that costs you nothing and protects you both.

12. Generative AI policy

You may use an AI assistant to explain Pthreads, to explain a compiler error, or to review code you have already written. You may **not** paste the assignment in and submit what comes back. If you use one, say so at the top of analysis.md in one line, naming the tool and what you used it for.

Note for your own sake: the four Part B bugs are exactly the bugs an AI assistant writes most often, because they are the bugs that are most common in the training data. Being able to spot them yourself is the point, and the midterm asks you to recognise and explain defects like these, so follow its own rules about predicting behaviour before you run anything.

13. Common pitfalls

1. **Passing `&i` to every thread.** All threads then read the same memory and `i` keeps changing. This is Bug 1, and it is the single most common Pthreads error. Give each thread its own struct.
2. **Reading results before joining.** Until `pthread_join` returns, the thread may not have run at all. This is Bug 2.

3. **Returning a pointer to a thread's local variable.** The thread's stack is gone when it exits. This is Bug 3.
4. **Checking pthread_create(...) == -1.** Pthreads returns the error number. This is Bug 4.
5. **lo = k * (n / nthreads).** The divide-before-multiply form loses the remainder, and the last elements never get summed. Your total will be slightly too small and you will not see why. Multiply first.
6. **Accumulating straight into arg->partial** in the inner loop. Correct but slower. Use a local.
7. **Timing a debug build.** -O2 is in the Makefile for a reason; -O0 timings tell you nothing about anything.
8. **Timing on battery, or with a browser open.** Use --reps and close things.